Android Enterprise Enrollment Troubleshooting — Managed Google Play, Web Enrollment & "Can't Add a Google Account" (AMAPI)

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Android Enterprise Enrollment Troubleshooting (Managed Google Play, Web Enrollment & "Can't Add a Google Account") 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 6

A note on evolving behavior . . . 6

Outcome . . . 6

1. Solution Overview — Android Enterprise Enrollment Troubleshooting (Managed Google Play, Web Enrollment & "Can't Add a Google Account") 6

What this solution is . . . 6

The enrollment moving parts (what the tree walks) . . . 6

The diagnostic method . . . 7

The named artifacts (use these names everywhere) . . . 7

What you can control . . . 8

Benefits . . . 8

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 9

2. Requirements Guide — Android Enterprise Enrollment Troubleshooting 9

R1. A reproducible test device you can wipe and re-enroll . . . 9

R2. Visibility into the device enrollment network (proxy, firewall, TLS inspection) . . . 10

R3. The tenant's binding model identified (account enterprise vs Google Workspace) . . . 10

R4. Read scopes for the diagnostic tools (least privilege) . . . 10

R5. Access to a comparable dev tenant/config for dev-vs-prod isolation (when that is the symptom) 11

R6. A change window and rollback plan for high-impact repairs . . . 11

Requirements summary . . . 12

Global Requirements — Microsoft Intune Android (Android Enterprise) Solutions 12

1. Microsoft 365 tenant with Intune and Android Enterprise . . . 13

2. The managed Google Play binding (account enterprise OR Google Workspace) . . . 13

3. Administrative access (least-privilege Intune roles) . . . 13

4. Enrollment method decision and Graph/PowerShell toolchain . . . 13

5. Network reachability to Google & Microsoft endpoints . . . 14

6. A pilot scope and change control . . . 14

3. Diagnostic Playbook — Android Enterprise Enrollment (symptom → root cause → fix) 15

Step 0 — Reproduce and route . . . 15

Playbook A — Managed Google Play binding / connector health . . . 16

Playbook B — Web / account-driven enrollment works in DEV but fails in PROD . . . 17
Playbook C — Enrollment-method path (wrong/failed provisioning for a method) . . . 18
Playbook D — Network / endpoint / proxy blocks . . . 18
Playbook E — Failure after an AMAPI migration . . . 19
Playbook F — Identity / account model mismatch ("can't add a Google account") . . . 20
Expand — from fixed device to fixed fleet . . . 21
5. Testing Guide — Android Enterprise Enrollment Troubleshooting 21
A. Environment validation (run once, before diagnosing) . . . 21
B. Binding / connector (Playbook A) . . . 22
C. Enrollment-method path (Playbook C) . . . 22
D. Network / endpoints (Playbook D) . . . 22
E. Dev-vs-prod (Playbook B) . . . 22
F. Account add (Playbook F) — the headline symptom . . . 23
G. Post-AMAPI (Playbook E) . . . 23
H. Fix-confirmation gates (must pass before expanding) . . . 23
Exit criteria (ready to expand) . . . 24
6. Troubleshooting Guide — the tools and the tree itself 24
Connecting and permissions . . . 24
Binding-health tool (`ANDROIDDX-BindingHealth.ps1`) . . . 24
Readiness / diagnostic-report tools . . . 25
Endpoint-reachability / config-diff tool . . . 25
Reproduction workflow . . . 26
When to escalate . . . 26
7. Best Practices — Android Enterprise Enrollment Troubleshooting 27
Diagnose in the right order . . . 27
Respect the account model . . . 27
Treat the network as a first-class suspect . . . 28
Handle high-impact changes with care . . . 28
Dev-vs-prod discipline . . . 28
Make it repeatable and reduce ticket volume . . . 28
Automation hygiene . . . 29
8. Deliverables Summary — Android Enterprise Enrollment Troubleshooting 29
Documentation . . . 29
PowerShell scripts (Microsoft Graph) . . . 29
Named artifacts produced . . . 30
What the buyer receives . . . 30

Outcome 30
Appendix A — PowerShell Scripts 31

Executive Summary — Android Enterprise Enrollment Troubleshooting (Managed Google Play, Web Enrollment & "Can't Add a Google Account")

Android Enterprise enrollment fails in maddening ways: a device won't **add a Google account** to Gmail or the device, **web / account-driven enrollment** works in a dev tenant but stalls in production, enrollment breaks **after an AMAPI migration**, or the managed Google Play **binding/connector** errors and apps never deliver. The cause is rarely where the error appears — it sits one or two layers down. This solution is a **layered enrollment diagnostic playbook**: a symptom-to-root-cause decision tree that walks the moving parts in the order that isolates a fault fastest, with the exact check and the exact fix at each node.

This is a complete, deploy-it-today package for turning a recurring class of enrollment tickets into a repeatable triage, written so an Intune administrator can follow it without prior Android-Enterprise troubleshooting experience.

What you get

• A plain-language **Solution Overview** of the enrollment moving parts — managed Google Play binding, the

enrollment-method path, network endpoints, the identity/account-add flow, dev-vs-prod configuration, and AMAPI vs legacy — and the Reproduce → Isolate → Resolve method.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite, on top of the

category-wide Global Requirements.

• A **Diagnostic Playbook** — a symptom-to-root-cause tree with six nodes (binding, dev-vs-prod, method path,

endpoints, post-AMAPI, account model), each with exact checks in the Intune admin center, Microsoft Graph, device logs, and endpoint reachability, plus a pass/fail gate and a rollback.

• **Five PowerShell (Microsoft Graph) tools**: a read-only enrollment-readiness assessment, a managed-Google-

Play binding health checker, an endpoint-reachability + dev-vs-prod config-diff helper, a symptom-to-cause diagnostic report, and a `-WhatIf`-gated scope-group helper. Read-only by default; no credentials in code.

• **Testing and Troubleshooting** guides with the fix-confirmation gates that tell you a symptom is actually

resolved before you expand to the fleet.

What this delivers

A named, repeatable workflow: a `Binding Health – Android – Baseline`, an `Enrollment Readiness – Android – Assessed`, an `Endpoint Allowlist – Android – Verified`, a `Diagnostic Report – Android – <symptom>`, and a fix proven on a reproduction device and an `Enrollment Scope – Android – Pilot` group before it touches the fleet — so devices enroll, Google accounts add, and the ticket class stops recurring.

The deployment model

Assess to Design to Configure to Pilot to Validate to Expand.

A note on evolving behavior

The Intune, Graph, managed Google Play, and Google Admin steps are authored from Microsoft's and Google's public documentation. The **AMAPI transition, web/account-driven enrollment, and whether a change forces device re-enrollment are actively evolving** — the solution flags every such point to **verify against current Microsoft guidance / your tenant** rather than asserting a behavior that may change.

Outcome

Android enrollments that stall — accounts that won't add, dev-works-prod-fails, post-AMAPI breakage — diagnosed by a repeatable decision tree to root cause and fixed, with fewer escalations and a lower enrollment-failure ticket volume.

1. Solution Overview — Android Enterprise Enrollment Troubleshooting (Managed Google Play, Web Enrollment & "Can't Add a Google Account")

What this solution is

Android Enterprise enrollment fails in maddening, hard-to-narrow ways: a device won't **add a Google account** to Gmail or to the device, **web / account-driven enrollment** works cleanly in a dev tenant but stalls in production, enrollment breaks **after an Android Management API (AMAPI) migration**, or the managed Google Play **binding/connector** throws errors and apps never deliver. The cause is almost never where the error appears — it lives one or two layers down, in the binding, the enrollment-method path, the network endpoints, or the account model.

This solution is a **layered enrollment diagnostic playbook**: a symptom-to-root-cause decision tree that walks the moving parts in the order that isolates a fault fastest — managed Google Play binding health, the enrollment-method path, network/endpoint reachability, the identity / account-add flow, dev-vs-prod configuration differences, and post-AMAPI-migration issues — with the exact check and the exact fix at each node. It gets devices enrolling and Google accounts adding, and it turns a recurring class of tickets into a repeatable 20-minute triage.

> ***A note on what's stable and what's evolving.*** *The Intune admin center, Microsoft Graph, managed Google Play, and the Google Admin console steps here are authored from Microsoft's and Google's public documentation. The **AMAPI transition** (Intune's move off the legacy Play EMM API), **web / account-driven enrollment**, and **whether a given change forces device re-enrollment** are **actively evolving** — every such point is flagged **"verify against current Microsoft guidance / your tenant"** rather than asserted.*

The enrollment moving parts (what the tree walks)

1. **Managed Google Play binding** — Intune's connection to managed Google Play, in one of two mutually

exclusive models: a **managed Google Play account enterprise** (tenants without Google Workspace) or the **Google Workspace / Cloud Identity ↔ Intune third-party EMM** integration (tenants with Workspace). A broken, unbound, or unsynced binding is the single most common upstream cause.

1. **Enrollment-method path** — **personally-owned work profile (BYOD)**, **fully managed**,

corporate-owned work profile, or **dedicated / COSU (kiosk)**. Each has a different provisioning path (managed account add vs QR/token/NFC/zero-touch device-owner) and therefore a different failure surface.

1. **Network / endpoint reachability** — the device's path to the Google (managed Google Play, Android) and

Microsoft (Intune, Entra) endpoints. A proxy, firewall, or TLS-inspection rule that blocks one host is a classic "can't add a Google account" root cause.

1. **Identity / account-add flow** — on a work profile the Google account is a **managed** account added by

the profile, not a consumer Gmail login; mismatching that model produces confusing "can't add account" symptoms.

1. **Dev-vs-prod configuration** — the setting, endpoint egress, Conditional Access rule, or binding model

that differs between a working dev tenant and a failing prod tenant.

1. **AMAPI vs legacy** — the state of the AMAPI transition and whether a migration regenerated a token,

changed the management path, or (verify) required re-enrollment.

The diagnostic method

Three words drive every investigation: **Reproduce. Isolate. Resolve.**

- **Reproduce** the failure on a known **reproduction device** so you are debugging a fact, not a report.
- **Isolate** by walking the tree top-down — binding first (it sits above everything), then the enrollment

path, then endpoints, then the account model — changing one variable at a time.

- **Resolve** at the node the tree lands on, then re-run the same reproduction to confirm the fix, and only

then expand back to the fleet.

The named artifacts (use these names everywhere)

Name each working artifact with the pattern `<Purpose> – <Scope> – <Stage>` and reuse the names verbatim across every document, script, and group:

- `Binding Health – Android – Baseline` — the managed Google Play connector state (from `ANDROIDDX-BindingHealth.ps1`).
- `Enrollment Readiness – Android – Assessed` — the per-tenant readiness table (from `ANDROIDDX-EnrollmentReadiness.ps1`).

• `Endpoint Allowlist – Android – Verified` — the reachability result from the device network.

• `Config Baseline – Android – Dev` / `Config Baseline – Android – Prod` — the two snapshots diffed for dev-vs-prod drift.

• `Diagnostic Report – Android – <symptom>` — the symptom-to-cause report for the failure under investigation.

• `Repro Device – Android – Test` — the dedicated wipe-and-retry reproduction device.

• `Enrollment Scope – Android – Pilot` / `Enrollment Scope – Android – Expanded` — the Entra ID groups that bound each pass.

What you can control

• Which symptom you are triaging (the tree has a dedicated entry per reported failure).

• The reproduction scope (the pilot group and the reproduction device).

• Which binding model the tenant uses (account enterprise vs Google Workspace) — chosen deliberately, changed only under change control.

• The endpoint allowlist and TLS-inspection exemptions on the device network.

• The enrollment method(s) in scope, and the change window for any high-impact change.

Benefits

• **Faster triage** — a repeatable tree replaces guesswork; most failures root-cause in one short session.

• **Fewer tickets** — the same playbook prevents the recurring "can't add a Google account" and "dev works, prod fails" tickets from recurring.

• **Safer changes** — binding, connector, and AMAPI-migration changes are treated as high-impact, with a pilot gate and a rollback before they touch the fleet.

• **Auditability** — named artifacts (binding health, readiness, endpoint allowlist, diagnostic report) make every finding and fix traceable.

Limitations (be honest with buyers)

• This is a **diagnostic** solution. It root-causes and fixes enrollment failures; it does not stand up the binding from scratch or design a greenfield enrollment program (those are separate solutions in the line).

• **AMAPI transition and re-enrollment behavior are evolving.** Where a step depends on the current AMAPI state or on whether a change forces re-enrollment, the solution flags it to **verify against current Microsoft guidance and your tenant** — it does not assert a behavior that Microsoft may change.

• **OEM and non-GMS specifics are pattern-level.** Samsung Knox, Zebra, and other rugged/kiosk devices, and

devices without Google Mobile Services (e.g. some China-market hardware), are covered at the pattern level; their consoles and firmware behaviors are deferred to the vendor's documentation.

- **On-device and network actions happen off-tenant.** Endpoint allowlisting, TLS-inspection exemptions, and

reading on-device logs happen on the network and device; the PowerShell tools cover the Intune/Graph side and the reachability probe.

- The **authoritative endpoint lists** are Microsoft's and Google's — the tool ships a documented core set

and tells you to verify against the current published lists for your region/tenant.

The standard deployment model

> *Assess the binding and readiness, design the isolation path for the reported symptom, configure the fix on a reproduction device / pilot scope, validate that the device enrolls and the account adds, then expand the fix to the fleet. The repeatable way: Assess to Design to Configure to Pilot to Validate to Expand.*

2. Requirements Guide — Android Enterprise Enrollment Troubleshooting

These are the prerequisites **specific to running this diagnostic playbook**. They sit on top of the **Global Requirements (Android)** published once for the whole category — a Microsoft 365 tenant with Intune and Android Enterprise, the managed Google Play binding (account enterprise OR Google Workspace), least-privilege Intune admin roles, a decided enrollment method with Graph PowerShell, network reachability to Google/Microsoft endpoints, and a pilot scope with change control. Verify those first; this guide adds what a troubleshooting engagement needs beyond them, each with **how to verify** and **how to implement if missing**.

> ***Scope note.*** *This solution assumes the binding and enrollment program already exist and are *failing* — it diagnoses and repairs them. If the managed Google Play connection has never been created, stand it up with the connection solution first, then return here to troubleshoot.*

R1. A reproducible test device you can wipe and re-enroll

- **Description:** A dedicated `Repro Device – Android – Test` — a physical or supported virtual Android

device you are free to **factory reset and re-enroll repeatedly** — matching the OEM, OS version, and enrollment method of the failing fleet as closely as possible.

- **Why required:** The method is Reproduce → Isolate → Resolve. You cannot isolate a fault you cannot

reproduce on demand, and you must not test fixes on a user's production device.

- **How to verify:** Confirm you have a device you can wipe, its OS build number, and whether it has Google

Mobile Services (GMS). Confirm it matches the failing population's OEM/OS/method.

• **How to implement if missing:** Obtain a representative device (borrow one from the affected population if

possible), record its build, and confirm GMS presence. For non-GMS/China-market hardware, note it — the account-add flow differs.

R2. Visibility into the device enrollment network (proxy, firewall, TLS inspection)

• **Description:** Knowledge of, and read visibility into, the **network path the device enrolls on** — the

proxy/firewall in front of it and whether **TLS/SSL inspection** is applied to Google/Microsoft hosts.

• **Why required:** Blocked endpoints and TLS interception of Google hosts are among the most common

"can't add a Google account" and stalled-enrollment root causes; you must be able to test reachability from the same network the device uses and to identify inspection rules.

• **How to verify:** Run `ANDROIDDX-EndpointReachability.ps1` from the device's network segment; confirm you

know who owns the proxy/firewall and can request an allowlist/exemption change.

• **How to implement if missing:** Establish a contact for the network team and a path to test from the

device network (or on the device's Wi-Fi). Obtain the current proxy config and TLS-inspection policy.

R3. The tenant's binding model identified (account enterprise vs Google Workspace)

• **Description:** A clear statement of **which of the two mutually exclusive managed Google Play models** the

tenant uses: **managed Google Play account enterprise** (no Google Workspace) or **Google Workspace / Cloud Identity ↔ Intune third-party EMM**.

• **Why required:** The two models fail — and are repaired — differently; the account-add flow and the

binding-owner account differ. Diagnosing the wrong model wastes the session.

• **How to verify:** Intune admin center → **Devices > Android > Android enterprise** (Connection/Enrollment)

— read the connection and owner account; `ANDROIDDX-BindingHealth.ps1` reports `OwnerUserPrincipalName` and `BindStatus`.

• **How to implement if missing:** Confirm with the tenant whether Google Workspace is in use; record the

model in `Binding Health – Android – Baseline`. Do **not** switch models to troubleshoot — that is a high-impact change under change control.

R4. Read scopes for the diagnostic tools (least privilege)

• **Description:** The **read-only Microsoft Graph scopes** the tools need:

`DeviceManagementServiceConfig.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`; plus `Group.ReadWrite.All` and `User.Read.All` **only** for the scope-group helper.

• **Why required:** Discovery, binding-health, and diagnostic-report tools read binding/enrollment/device

state; only the scope helper writes (group membership), and it is `-WhatIf`-gated.

• **How to verify:** `Connect-MgGraph` succeeds with the read scopes; `ANDROIDDX-EnrollmentReadiness.ps1`

runs without a permissions error.

• **How to implement if missing:** Have an admin consent the read scopes for the account running the tools.

Grant the group-write scope only when you run the scope helper.

R5. Access to a comparable dev tenant/config for dev-vs-prod isolation (when that is the symptom)

• **Description:** When the reported symptom is **"works in dev, fails in prod,"** access to the **working dev

tenant's configuration** (or an exported snapshot of it) to diff against prod.

• **Why required:** The dev-vs-prod node isolates by **diffing** the two configurations; without the working

reference you cannot see the drift.

• **How to verify:** You can export a config snapshot from both tenants with `ANDROIDDX-DiagnosticReport.ps1

-JsonPath and diff them with ANDROIDDX-EndpointReachability.ps1 -CompareConfig`.

• **How to implement if missing:** Obtain read access (or a snapshot export) from whoever owns the dev tenant.

If prod is the only environment, isolate against the documented expected configuration instead.

R6. A change window and rollback plan for high-impact repairs

• **Description:** An approved **change window** and a written **rollback** for any repair that touches the

binding, the connector, an enrollment profile/token, or an AMAPI-related setting — because these are high-impact and may affect **existing** enrolled devices.

• **Why required:** Re-establishing a binding, regenerating a token, or an AMAPI-related change can affect

more than the one device under test; the pilot-first model and a rollback protect the fleet. Whether such a change forces **re-enrollment** is version-dependent — **verify against current Microsoft guidance / your tenant** before committing.

• **How to verify:** A named approver, a change window, and a documented rollback exist before any fleet-wide

or connector-level change.

• **How to implement if missing:** Agree change control and write the rollback (e.g. how to restore the prior

enrollment profile / binding state) before you apply anything beyond the reproduction device.

Requirements summary

#	Requirement	Verify	Implement if missing
R1	Reproducible wipe-and-retry test device	Device you can factory reset; OS build & GMS known; matches fleet	Obtain a representative device; record build/GMS
R2	Device-network visibility (proxy/TLS)	Reachability test runs from device network; proxy/TLS owner known	Get network-team contact and current proxy/TLS policy
R3	Binding model identified	Intune Android enterprise page / `BindingHealth` owner & status	Confirm Workspace use; record model; don't switch to troubleshoot
R4	Least-privilege read scopes	`Connect-MgGraph` + readiness tool run clean	Admin-consent the read scopes; group-write only for scope helper
R5	Dev config/snapshot for dev-vs-prod	Snapshot export + `-CompareConfig` diff possible	Obtain dev read access/snapshot, or diff vs expected config
R6	Change window + rollback for high-impact repairs	Named approver, window, written rollback	Agree change control; write rollback before connector/AMAPI change

Global Requirements — Microsoft Intune Android (Android Enterprise) Solutions

These requirements apply to **every** solution in the **Android** category (Android Enterprise connection & managed Google Play, Google Workspace ↔ Intune EMM integration, the Android Management API (AMAPI) migration, dedicated/COSU & work-profile enrollment, app management, and enrollment troubleshooting). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Android solution).*** *The **Microsoft Intune admin center** (intune.microsoft.com), **Microsoft Graph**, the **managed Google Play** enrollment surface, and the **Google Admin console** (for Google Workspace / Cloud Identity integration) are authored at exact, current click-by-click / cmdlet detail from Microsoft's and Google's public documentation. The **Android Management API (AMAPI)** transition (Intune's move off the legacy Play EMM API) and **web-based /***

account-driven enrollment *are* ***actively evolving*** *— steps, defaults, and whether a change triggers* ***re-enrollment*** *are* ***cited or flagged "verify against current Microsoft guidance / your tenant"****, never asserted from memory or invented.* ***OEM specifics*** *(Samsung Knox, Zebra, and other rugged/kiosk vendors) and* ***third-party apps*** *are integrated at the pattern level; their consoles are deferred to the vendor's docs. Automation is* ***PowerShell*** *(Microsoft Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with Intune and Android Enterprise

• **Description:** A Microsoft 365 tenant with **Microsoft Intune** and Android Enterprise available, and the licensing the capability requires.

• **Why required:** All Android policy, enrollment, and app configuration is performed in Intune.

• **How to verify:** Sign in to the Intune admin center; confirm Android platform availability.

• **How to implement if missing:** Provision/assign Intune licensing before starting.

2. The managed Google Play binding (account enterprise OR Google Workspace)

• **Description:** Intune connected to **managed Google Play** — either the **managed Google Play account enterprise **(for tenants WITHOUT Google Workspace) or the** Google Workspace / Cloud Identity ↔ Intune third-party EMM** integration (for tenants that use Google Workspace). One and only one model applies per tenant.

• **Why required:** Android Enterprise enrollment and app delivery depend on the managed Google Play connection; the two binding models are mutually exclusive and must be chosen deliberately.

• **How to verify:** Intune admin center → **Devices > Android > Android enterprise (or Enrollment)** — confirm the connection status and which model is in place.

• **How to implement if missing:** Establish the correct binding (this is often the first solution in the line) before enrolling devices.

3. Administrative access (least-privilege Intune roles)

• **Description:** Access with the least-privilege role the solution names — typically **Intune Administrator**, with** Global Administrator** only where genuinely required (e.g. the initial managed Google Play / Google Workspace binding).

• **Why required:** Enrollment, connector, and app changes require the corresponding role.

• **How to verify:** Confirm the role in the M365 admin center / Entra roles.

• **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

4. Enrollment method decision and Graph/PowerShell toolchain

• **Description:** A decided **enrollment method** — personally-owned **work profile** (BYOD), **fully managed**, corporate-owned work profile**, or** dedicated / COSU (kiosk) **— plus a workstation with** Microsoft Graph PowerShell** for automation.

• **Why required:** Profiles, restrictions, and app assignment differ by enrollment method; the provided scripts run through Graph.

• **How to verify:** Confirm the target enrollment method(s) with the device owners; `Connect-MgGraph` succeeds.

• **How to implement if missing:** Choose the enrollment method and install Graph PowerShell before starting.

5. Network reachability to Google & Microsoft endpoints

• **Description:** Devices and the enrollment path can reach the required **Google (managed Google Play, Android) **and** Microsoft (Intune, Entra)** service endpoints (no proxy/filter blocks the enrollment or account sign-in).

• **Why required:** Enrollment, managed Google Play account provisioning, and adding a Google account all require reachability; blocked endpoints are a common cause of "can't add a Google account" failures.

• **How to verify:** Confirm the required endpoints are allowed on the device network.

• **How to implement if missing:** Allow the documented Google/Microsoft endpoints before enrolling.

6. A pilot scope and change control

• **Description:** A **pilot group / small device set** to validate on before fleet-wide rollout, and a **change-control** process for production changes (enrollment-model, connector, and AMAPI-migration changes are high-impact and may affect existing devices).

• **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and protects the production fleet.

• **How to verify:** Confirm a named pilot scope and an approval/change window are available.

• **How to implement if missing:** Create a pilot scope and agree change control before production changes.

*The deployment model across every Android solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (Microsoft Graph) — never with hardcoded credentials or secrets. Where the AMAPI transition or enrollment behavior is version-dependent, verify against current Microsoft guidance rather than assuming.*

3. Diagnostic Playbook — Android Enterprise Enrollment (symptom → root cause → fix)

This is a **symptom-to-root-cause decision tree**. Start at **Step 0** to reproduce and route, then follow the one **Playbook node** the routing table sends you to. Each node gives the **exact checks** (Intune admin center, Microsoft Graph, device logs, endpoint reachability) and the **fix**, with a **pass/fail gate** and, where the fix is high-impact, a **rollback**. Work **one variable at a time** and confirm each fix by re-running the same reproduction before moving on.

> ***Golden rules.*** *(1)* ***Reproduce before you change anything*** *— debug a fact, not a report. (2)* ***Binding sits above everything*** *— clear Playbook A before chasing anything else. (3)* ***Pilot first*** *— prove every fix on* `Repro Device – Android – Test` *and the* `Enrollment Scope – Android – Pilot` *group before the fleet. (4)* ***AMAPI / re-enrollment behavior is evolving*** *— where flagged,* ***verify against current Microsoft guidance / your tenant****; do not assume.*

Step 0 — Reproduce and route

0.1 Reproduce on the test device. Factory reset `Repro Device – Android – Test`, connect it to the **same network** the failing fleet uses, and re-run the failing enrollment with the exact method the fleet uses (BYOD work profile via Company Portal, or the device-owner QR/token/NFC/zero-touch path). Capture the **exact on-device error text and screen**, the OS build, and whether the device has Google Mobile Services.

0.2 Establish the tenant baseline (read-only, changes nothing).

- `powershell -File .\ANDROIDDX-EnrollmentReadiness.ps1 -Days 14 -CsvPath .\android-readiness.csv` → record as `Enrollment Readiness – Android – Assessed`.
- `powershell -File .\ANDROIDDX-BindingHealth.ps1 -JsonPath .\binding.json` → record as `Binding Health – Android – Baseline`.
- `powershell -File .\ANDROIDDX-DiagnosticReport.ps1 -Symptom <symptom> -JsonPath .\prod-config.json` → record as `Diagnostic Report – Android – <symptom>`.

0.3 Route by symptom.

Reported symptom	Start at	Also check
"Can't add a Google account" to Gmail/the device	**F** (account model)	**A** (binding), **D** (endpoints)
Web / account-driven enrollment works in DEV but fails in PROD	**B** (dev-vs-prod)	**D** (endpoints), **A** (binding)
Enrollment broke **after an AMAPI migration**	**E** (post-AMAPI)	**A** (binding)
Managed Google Play binding / connector error; apps won't deliver	**A** (binding)	—

Reported symptom	Start at	Also check
Enrollment stalls / times out at a network step	**D** (endpoints)	**A** (binding)
Wrong/failed provisioning for a specific enrollment method	**C** (method path)	**A** (binding)

> ***If the binding baseline in 0.2 is not*** `bound` ***/ not synced, do Playbook A first regardless of symptom*** *— a broken connector produces almost every downstream symptom.*

Playbook A — Managed Google Play binding / connector health

Symptom: binding/connector error, apps not delivering, or any symptom while `bindStatus ≠ bound`.

Checks

1. **Graph / tool:** `ANDROIDDX-BindingHealth.ps1` — read `BindStatus`, `OwnerUserPrincipalName`,

`LastAppSyncStatus`, `LastAppSyncAgeHours`, and the enabled enrollment paths.

1. **Intune admin center: Devices > Android > Android enterprise** (Connection/Enrollment) — confirm the

connection shows connected, note the **binding model** (account enterprise vs Google Workspace) and the **owner account**.

1. **Model check:** confirm exactly **one** model is in place. A tenant with Google Workspace must use the

Workspace / third-party-EMM binding; a tenant without it uses the account-enterprise binding.

Fix (decision)

• `bindStatus = bound`, owner present, sync recent + `success` → **binding is healthy; return to your routed

node.**

• `bindStatus ≠ bound` **or** no owner account → the connector is broken. **Repair / re-establish the managed

Google Play connection **(Global Admin, one-time Google consent) per the connection setup process. This is** high-impact** — see rollback.

• Sync stale/failed → force a managed Google Play **sync** and re-check `LastAppSyncStatus`; if it stays

non-`success`, treat as a broken connection.

Pass/fail gate: `ANDROIDDX-BindingHealth.ps1` reports `bound`, an owner account, and a recent `success` sync. **Fail** → do not proceed; the binding must be healthy first.

Rollback (high-impact): before re-establishing a binding, record the current connection state and owner account. Re-establishing or unbinding the connection **can affect existing enrolled devices — whether it forces re-enrollment is version-dependent; verify against current Microsoft guidance / your tenant**

— so do it only in an approved change window with the connection-setup solution's rollback in hand.

Playbook B — Web / account-driven enrollment works in DEV but fails in PROD

Symptom: the same web / account-driven (AMAPI) enrollment succeeds in a dev tenant and fails in prod.

Checks

1. **Config diff:** export a snapshot from each tenant with `ANDROIDDX-DiagnosticReport.ps1 -JsonPath` and

diff them: `ANDROIDDX-EndpointReachability.ps1 -CompareConfig -DevJson .\dev-config.json -ProdJson .\prod-config.json`. Record the differing settings as `Config Baseline – Android – Dev` vs `Config Baseline – Android – Prod`.

1. **Endpoint egress:** run `ANDROIDDX-EndpointReachability.ps1` **from the prod device network** and again

from the dev network — an endpoint reachable in dev but **Blocked** in prod is the answer (go to **D**).

1. **Enrollment restriction / Conditional Access:** Intune **Devices > Enrollment > Enrollment restrictions**

and Entra **Conditional Access** — compare the policies that apply to the prod enrolling user/device vs dev (platform blocked, personally-owned blocked, a CA grant requiring compliance/MFA that the fresh device can't satisfy).

1. **Binding model / consent:** confirm prod's managed Google Play binding is fully consented and `bound`

(Playbook A) — a dev tenant is often bound while prod's binding is partial.

1. **Web/account-driven specifics:** the account-driven enrollment method is **evolving** — confirm the prod

tenant meets the **current** prerequisites for it (**verify against current Microsoft guidance**), since dev may have been configured under different/newer settings.

Fix

• Reconcile the specific setting the diff surfaced (bring prod to match the known-good dev value **only after

confirming that value is correct for prod**).

• Allowlist the endpoint that is blocked in prod (Playbook D).

• Relax/scope the prod enrollment restriction or CA rule that blocks the fresh device, under change control.

Pass/fail gate: the prod reproduction device now completes web/account-driven enrollment. **Fail** → the remaining diff items or an endpoint block are still in play; continue isolating one variable at a time.

Rollback: enrollment-restriction and CA changes are reversible — record the prior policy state before editing so you can restore it.

Playbook C — Enrollment-method path (wrong/failed provisioning for a method)

Symptom: provisioning fails or lands in the wrong state for a **specific** enrollment method.

Checks (per method)

- **Personally-owned work profile (BYOD):** enrollment is via **Company Portal**; the work profile is

created and a **managed** Google account is added by the profile. Confirm Company Portal is current and the user is licensed/allowed.

- **Fully managed / dedicated (COSU) / corporate-owned work profile:** these use a **device-owner** path

(QR code, token, NFC, or zero-touch) from a **fresh/factory-reset** device. Confirm an `androidDeviceOwnerEnrollmentProfile` exists (readiness tool reports the count) and the QR/token is current and not expired.

- **Intune admin center: Devices > Enrollment > Android** — confirm the profile for the method exists and

is assigned to `Enrollment Scope – Android – Pilot`.

- **Device logs:** capture the on-device provisioning error; a device-owner path started on a

non-factory-reset device is a common, self-inflicted failure.

Fix

- Match the method to its path: device-owner methods require a **factory-reset** device and a valid

profile/token; BYOD requires Company Portal. Regenerate an expired QR/token. Assign the profile to the pilot scope with `ANDROIDDX-Set-ScopeTargeting.ps1`.

Pass/fail gate: the device provisions into the **intended** method state (work profile vs fully managed vs dedicated). **Fail** → re-check the profile assignment and the device's factory-reset state.

Rollback: regenerating a token invalidates the prior one — record it first; profile assignment changes are reversible via the scope group.

Playbook D — Network / endpoint / proxy blocks

Symptom: enrollment stalls at a network step, or "can't add a Google account" with the binding healthy.

Checks

1. **Reachability from the device network:** `ANDROIDDX-EndpointReachability.ps1 -CsvPath .\endpoints.csv` —

record as `Endpoint Allowlist – Android – Verified`. Any **Blocked** host is a root cause.

1. **TLS/SSL inspection:** confirm whether the proxy performs **TLS interception** on Google/Microsoft hosts

— intercepted TLS to `accounts.google.com` / `*.googleapis.com` breaks token/sign-in even when the host is "reachable."

1. **Authoritative lists:** the tool ships a **documented core** endpoint set — cross-check against the

current Microsoft (Intune network endpoints) and Google (Android Enterprise network requirements) published lists for your region/tenant; **verify these lists are current** before allowlisting.

1. **Device basics:** confirm device **date/time** is correct and **Google Play services** are present and

current (non-GMS/China-market devices behave differently — see **F**).

Fix

• **Allowlist** the blocked Google/Microsoft endpoints on the proxy/firewall and **exempt** Google/Microsoft

hosts from **TLS inspection**. Correct device date/time. Update Google Play services where applicable.

Pass/fail gate: every required endpoint reports **Reachable**, TLS is not intercepted on Google hosts, and the reproduction device now adds the account / completes enrollment. **Fail** → an endpoint or inspection rule is still blocking; re-run the reachability tool after each network change.

Rollback: allowlist/exemption changes are network-team changes — record the prior rule so they can be reverted if they conflict with security policy.

Playbook E — Failure after an AMAPI migration

Symptom: enrollment (or management) broke **after** an Android Management API (AMAPI) migration.

> ***This node is the most version-sensitive in the tree.*** *The AMAPI transition and* ***whether a migration forces device re-enrollment*** *are* ***actively evolving — verify every point below against current Microsoft guidance and your tenant****; do not assert the behavior from memory.*

Checks

1. **Re-enrollment requirement (verify):** confirm whether the specific AMAPI change in your tenant requires

affected devices to **re-enroll** — **verify against current Microsoft guidance / your tenant.** If it does, the "failure" may be expected until re-enrollment.

1. **Token / profile regeneration:** check whether the migration **regenerated** enrollment profiles or

tokens (Playbook C) — an old QR/token distributed before the migration will now fail.

1. **Binding / app re-sync:** run `ANDROIDDX-BindingHealth.ps1`; force a managed Google Play sync. Migration

can leave app approvals/config needing a re-sync (Playbook A).

1. **Diagnostic report:** `ANDROIDDX-DiagnosticReport.ps1 -Symptom AfterAmapiMigration` for the ranked

candidates.

Fix

• If re-enrollment is required (**verified**), **re-enroll the reproduction device**, confirm success, then

roll re-enrollment out via the pilot scope under change control.

- Redistribute **regenerated** QR/tokens; retire the old ones.
- Force the post-migration **app/config re-sync** and confirm `LastAppSyncStatus = success`.

Pass/fail gate: the reproduction device enrolls and is managed under the post-AMAPI path, and app sync is `success`. **Fail** → re-check the re-enrollment requirement and token freshness.

Rollback (high-impact): an AMAPI migration is not casually reversible — treat forward-fix (re-enroll / re-sync) as the path, keep the change window, and escalate to Microsoft support if the documented behavior does not match your tenant.

Playbook F — Identity / account model mismatch ("can't add a Google account")

Symptom: the device or Gmail won't add a Google account, or the added account is rejected.

Checks

1. **Account model:** determine which model applies. On a **work profile**, the Google account used for

managed Google Play is a **managed account added by the profile** — it is **not** a consumer Gmail login, and **manually adding a personal Google account inside the work profile is expected to fail/absent**. Confirm the user is trying the **right** flow for the enrollment method.

1. **Binding provisioning:** if the managed account never provisions, the binding is likely the cause →

Playbook A (`bindStatus`, owner account, sync).

1. **Endpoints / TLS:** `ANDROIDDX-EndpointReachability.ps1` — a blocked `accounts.google.com` /

`*.googleapis.com` / `play.google.com` or intercepted TLS blocks the account add → Playbook D.

1. **Device / GMS:** confirm **Google Play services** are present and current and the **date/time** is

correct. On **non-GMS / China-market** hardware the standard Google-account add path may not exist — this is a device/OEM constraint, not a tenant fault (pattern-level; defer to the vendor).

1. **Workspace identity match (Workspace-bound tenants):** if the tenant uses the Google Workspace binding,

confirm the user's identity is correctly represented in **Cloud Identity / Workspace** so the managed account can be issued.

Fix

- Use the **correct account flow** for the method (managed account via the profile/enrollment, not a manual

consumer Gmail add). Repair the binding (A) if the managed account never provisions. Clear the endpoint/TLS block (D). Update Google Play services / fix date-time. For non-GMS devices, follow the OEM's documented path or choose a supported device.

Pass/fail gate: the reproduction device successfully adds the **managed** Google account and managed Google Play is usable. **Fail** → step back to A (binding) or D (endpoints) — those two produce most

account-add failures.

Rollback: account-flow and device fixes are non-destructive; binding and network fixes carry their own rollbacks (A, D).

Expand — from fixed device to fixed fleet

Once the reproduction device passes the routed node's gate:

1. Apply the same fix to the `Enrollment Scope – Android – Pilot` group (build/refresh it with `ANDROIDDX-Set-ScopeTargeting.ps1 -WhatIf` then apply).
1. Re-run the reproduction on **two or three** pilot devices; confirm the gate passes for all.
2. Promote to `Enrollment Scope – Android – Expanded` and roll out under change control.
3. Re-run `ANDROIDDX-EnrollmentReadiness.ps1` and confirm the ticket class stops recurring.

*Follow the house model throughout: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, one variable at a time, with the pass/fail gate and rollback at each node.*

5. Testing Guide — Android Enterprise Enrollment Troubleshooting

Testing here proves two things: that the **diagnostic tools read the tenant correctly**, and that each **fix actually resolves the reproduced failure** at its pass/fail gate. Run the environment tests once, then the symptom tests for whichever failure you are triaging. Every test is **reproduce** → **check** → **gate**.

A. Environment validation (run once, before diagnosing)

#	Test	How	Pass
A1	Graph connectivity & read scopes	`ANDROIDDX-EnrollmentReadiness.ps1 -Days 14`	Runs clean; readiness table prints; no permission error
A2	Binding is readable	`ANDROIDDX-BindingHealth.ps1 -JsonPath .\binding.json`	Prints `BindStatus`/owner/sync; snapshot written
A3	Reachability tool runs from device network	`ANDROIDDX-EndpointReachability.ps1 -CsvPath .\endpoints.csv` on the device segment	Every required endpoint `Reachable`; CSV written
A4	Reproduction device ready	Factory reset `Repro Device – Android – Test`; note OS build + GMS	Device wipes and is ready to re-enroll
A5	Scope helper preview	`ANDROIDDX-Set-ScopeTargeting.ps1 -GroupName "Enrollment Scope - Android - Pilot" -MembersCsv .\pilot-users.csv -WhatIf`	Shows planned adds; writes nothing

B. Binding / connector (Playbook A)

#	Test	Pass
B1	Binding healthy	`ANDROIDDX-BindingHealth.ps1` reports `bound`, owner present, sync `success` and recent
B2	One model only	Exactly one of account-enterprise / Google Workspace binding is in place
B3	App sync recovers	After a forced sync, `LastAppSyncStatus = success`
Gate	**Binding gate**	**All of B1–B3 pass before diagnosing anything downstream**

C. Enrollment-method path (Playbook C)

#	Test	Pass
C1	Device-owner profile present	Readiness tool shows ≥1 `androidDeviceOwnerEnrollmentProfile` for fully managed/dedicated/corp-WP
C2	Correct method state	Reproduction device provisions into the intended state (work profile vs fully managed vs dedicated)
C3	Token valid	QR/token is current (not expired); a regenerated token enrolls

D. Network / endpoints (Playbook D)

#	Test	Pass
D1	Endpoints reachable	`ANDROIDDX-EndpointReachability.ps1` = all required hosts `Reachable` from the device network
D2	No TLS interception on Google hosts	Proxy does not intercept TLS to `accounts.google.com` / `*.googleapis.com` / `play.google.com`
D3	Device basics	Device date/time correct; Google Play services present/current (GMS devices)

E. Dev-vs-prod (Playbook B)

#	Test	Pass
E1	Snapshots exportable	`ANDROIDDX-DiagnosticReport.ps1 -JsonPath` produces a dev and a prod snapshot
E2	Diff surfaces drift	`-CompareConfig` lists the differing settings (or confirms none)
E3	Reconciled prod enrolls	After reconciling the surfaced difference, prod reproduction device enrolls

F. Account add (Playbook F) — the headline symptom

#	Test	Pass
F1	Correct flow used	The managed account is added by the profile/enrollment (not a manual consumer Gmail add)
F2	Managed account provisions	Reproduction device adds the **managed** Google account; managed Google Play is usable
F3	Endpoints clear	D1–D3 pass (account-add failures are usually binding or endpoint)

G. Post-AMAPI (Playbook E)

#	Test	Pass
G1	Re-enrollment requirement known	Whether the change forces re-enrollment is **verified against current Microsoft guidance / the tenant**
G2	Fresh tokens	Regenerated QR/tokens enroll; old ones retired
G3	Post-migration sync	`LastAppSyncStatus = success` after a forced sync

H. Fix-confirmation gates (must pass before expanding)

Symptom fixed	Gate
Can't add a Google account	Reproduction device adds the managed account; managed Google Play usable (F2)
Dev works / prod fails	Prod reproduction device completes web/account-driven enrollment (E3)

Symptom fixed	Gate
Binding/connector error	`bound` + owner + `success` sync (B1)
Endpoint block	All endpoints `Reachable`, no Google-host TLS interception (D1–D2)
Method provisioning	Device lands in the intended method state (C2)
Post-AMAPI	Reproduction device enrolls under the post-AMAPI path; sync `success` (G1–G3)

Exit criteria (ready to expand)

- The routed Playbook node's **pass/fail gate** is green on `Repro Device – Android – Test`.
- The fix passes on **two or three** `Enrollment Scope – Android – Pilot` devices.
- `ANDROIDDX-EnrollmentReadiness.ps1` shows PASS on the previously-failing checks.
- Any high-impact change (binding/connector/AMAPI) was applied in the **change window** with the **rollback**

recorded, and re-enrollment behavior was **verified**, not assumed.

6. Troubleshooting Guide — the tools and the tree itself

The Diagnostic Playbook (Section 3) troubleshoots your **tenant's** enrollment failures. This guide troubleshoots **running the solution** — the PowerShell tools, the Graph connection, and the reproduction workflow — so the diagnostic session itself doesn't stall.

Connecting and permissions

Symptom	Likely cause	Fix
`Connect-MgGraph` fails or prompts endlessly	SDK not installed, or admin consent missing for the scopes	Install the Microsoft Graph PowerShell SDK; have an admin consent the read scopes (`DeviceManagementServiceConfig.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`)
`403 / Authorization_RequestDenied` on a read	Missing one of the read scopes	Reconnect with all three read scopes; confirm the signing-in account holds the Intune-read role
Scope helper can't write group membership	Missing `Group.ReadWrite.All` / `User.Read.All`	Connect the scope helper with those scopes; run `-WhatIf` first to preview

Binding-health tool (`ANDROIDDX-BindingHealth.ps1`)

Symptom	Likely cause	Fix
`404` reading the binding settings	The managed Google Play connection has **never** been created	This solution troubleshoots an existing binding — stand it up with the connection solution first, then return
Properties show as blank/null	Beta resource shape differs in your tenant, or binding partially consented	**Verify property names against current Microsoft Graph documentation**; re-check the connection consent in Intune
`bindStatus` empty but Intune shows connected	Transient read, or model mismatch	Re-run; confirm the binding model (account-enterprise vs Workspace) in Intune > Devices > Android > Android enterprise

Readiness / diagnostic-report tools

Symptom	Likely cause	Fix
WARN on "Android enrollment restriction" but Android does enroll	The tool reads the presence of a platform-restriction policy, not its per-platform detail	Confirm in Intune > Devices > Enrollment > Enrollment restrictions that Android Enterprise is not `Blocked`; treat WARN as "go verify," not "broken"
"Recent Android enrollments = 0" but devices exist	No enrollments **in the window**, or older than `-Days`	Increase `-Days`; zero recent enrollments when you expect them is itself a symptom, not a tool error
`androidDeviceOwnerEnrollmentProfiles` empty	Tenant uses only BYOD work profile (no device-owner path)	Expected — device-owner profiles are only needed for fully managed / dedicated / corp-owned work profile
Diagnostic report lists causes that don't apply	Mapping is **heuristic guidance**, not an assertion	Use it to prioritize the tree; confirm each candidate against your tenant and current Microsoft guidance

Endpoint-reachability / config-diff tool

Symptom	Likely cause	Fix
Endpoint shows `Blocked` from an admin PC but device enrolls fine	You tested from the wrong network segment	Run the tool **from the device's own network/Wi-Fi** — egress rules differ by VLAN

Symptom	Likely cause	Fix
Everything `Reachable` but account add still fails	TLS/SSL **interception** on Google hosts (host is reachable, TLS is broken)	Exempt `accounts.google.com` / `*.googleapis.com` / `play.google.com` from TLS inspection
Reachable list looks incomplete for my region	The shipped list is a documented **core** set, not exhaustive	Cross-check and extend against the **current** Microsoft (Intune network endpoints) and Google (Android Enterprise network requirements) published lists
`-CompareConfig` shows "no differences" but prod still fails	The failure is outside captured config (endpoints, account model, device state, CA)	Move to Playbook D (endpoints), F (account model), or check Conditional Access

Reproduction workflow

Symptom	Likely cause	Fix
Fix "works" then fails on the next device	You changed more than one variable, or the device wasn't fully reset	Reset the test device fully; change **one** variable; re-run the reproduction each time
Device-owner enrollment won't start	Device not factory-reset, or expired QR/token	Factory-reset the device; regenerate the QR/token (Playbook C)
Can't add a "personal" Google account in the work profile	**Expected** — the work-profile account is a **managed** account added by the profile	Use the managed account flow; document this for the service desk (Playbook F)
Post-AMAPI device won't enroll	Change may require re-enrollment; old token stale	**Verify the re-enrollment requirement against current Microsoft guidance / your tenant**; re-enroll on the test device; redistribute fresh tokens (Playbook E)

When to escalate

- **Binding cannot be re-established / re-consented** even by a Global Admin, or `bindStatus` stays in error

after a documented repair → open a Microsoft support case; capture `Binding Health – Android – Baseline`.

- **Post-AMAPI behavior contradicts the current documentation** for your tenant → escalate to Microsoft

support with your `Diagnostic Report – Android – AfterAmapiMigration` and the change record.

- **An endpoint block is a security-policy decision** (TLS inspection required) → escalate to the network/

security owner with `Endpoint Allowlist – Android – Verified` to agree an exemption.

- **Non-GMS / OEM-specific device behavior** (China-market hardware, rugged/kiosk quirks) → defer to the OEM's

documentation; this solution covers those at the pattern level only.

> *Keep every finding as its named artifact so an escalation arrives with evidence, not a description — and so the next engineer inherits the trail.*

7. Best Practices — Android Enterprise Enrollment Troubleshooting

Field-proven habits that make enrollment failures fast to root-cause and rare to recur. They reinforce the tree in the Diagnostic Playbook and the pass/fail discipline in the Testing Guide.

Diagnose in the right order

- **Reproduce first, always.** Never troubleshoot from a screenshot or a ticket description alone — wipe the

test device and reproduce the exact failure on the exact network. A fault you can't reproduce is a fault you can't confirm you fixed.

- **Binding before everything.** `bindStatus ≠ bound`, a missing owner account, or a stale sync produces

almost every downstream symptom. Clear Playbook A before chasing account-add or dev-vs-prod.

- **One variable at a time.** Change one thing, re-run the reproduction, then decide. Batch changes hide which

one worked and which one broke something else.

- **Let the symptom pick the entry node,** but keep the two universal suspects — **binding** and

endpoints — in view for every symptom.

Respect the account model

- On a **work profile**, the Google account is a **managed account added by the profile** — not a consumer

Gmail login. Teach the service desk this: "can't add a personal Google account in the work profile" is usually **expected behavior**, not a fault.

- Match the **enrollment method to its path**: device-owner methods (fully managed / dedicated / corp-owned

work profile) need a **factory-reset** device and a valid QR/token; BYOD needs Company Portal. A device-owner flow started on a non-reset device is a self-inflicted failure.

Treat the network as a first-class suspect

- Test reachability **from the device's own network segment**, not from an admin workstation on a different

VLAN — they often have different egress rules.

- **Exempt Google and Microsoft hosts from TLS/SSL inspection.** Intercepted TLS to `accounts.google.com` /

`*.googleapis.com` breaks account add and enrollment even when the host is "reachable."

- Keep the **allowlist current** against Microsoft's and Google's **published** endpoint lists — they change.

The shipped tool carries a documented core set and tells you to verify; do verify, per region/tenant.

Handle high-impact changes with care

- **Binding, connector, enrollment-profile/token, and AMAPI changes can affect existing devices.** Always use

a change window and a written rollback, and prove the fix on the pilot scope first.

- **Never switch the binding model to troubleshoot.** Account-enterprise vs Google Workspace is a deliberate,

high-impact architectural choice — diagnose within the model the tenant already uses.

- **Do not assert re-enrollment behavior.** Whether an AMAPI change forces re-enrollment is **evolving** —

verify against current Microsoft guidance / your tenant every time, and plan for re-enrollment as a possibility, not a surprise.

Dev-vs-prod discipline

- When "it works in dev," **diff the two configs** before theorizing — export both snapshots and compare.

Most "prod is haunted" tickets are a single setting, endpoint, or Conditional Access rule that differs.

- Keep a **known-good dev/reference config** so you always have something to diff prod against.

Make it repeatable and reduce ticket volume

- Keep a **standing reproduction device** and a **pilot scope group** so every retest targets a known set.
- **Record every finding** as its named artifact (`Binding Health – Android – Baseline`,

`Endpoint Allowlist – Android – Verified`, `Diagnostic Report – Android – <symptom>`) so the next engineer inherits the trail.

- Turn each root cause into a **service-desk note** (especially the account-model one) so the same class of

ticket is answered at tier 1 next time instead of escalating.

- Run `ANDROIDDX-EnrollmentReadiness.ps1` **periodically**, not just during incidents — a stale sync or a

drifting binding is cheaper to catch before it becomes a wave of failed enrollments.

Automation hygiene

- Keep the tools **read-only by default**; only the scope helper writes, and it is `-WhatIf`-gated and

idempotent. Preview with `-WhatIf` before applying.

- Connect with **least-privilege scopes**; never put credentials or secrets in scripts.
- Where a Graph resource is **beta** (the managed-store binding settings), treat property names as subject to

change and **verify against current Microsoft Graph documentation**.

8. Deliverables Summary — Android Enterprise Enrollment Troubleshooting

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The enrollment moving parts, the named artifacts, the Reproduce→Isolate→Resolve method, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (on top of the Global Requirements)
Global Requirements (Android)	Category-wide prerequisites, published once
3. Diagnostic Playbook	Symptom→root-cause decision tree: Step 0 routing + Playbooks A–F, each with exact checks, fix, pass/fail gate, rollback
5. Testing Guide	Environment + per-symptom tests and the fix-confirmation gates
6. Troubleshooting Guide	Field issues with the tools and the tree, and remediation
7. Best Practices	Field-proven recommendations for fast root-cause and lower ticket volume

PowerShell scripts (Microsoft Graph)

Script	What it does
`ANDROIDDX-EnrollmentReadiness.ps1`	Read-only. Per-tenant readiness table (binding, enrollment restriction, profiles, recent Android enrollments) with PASS/WARN
`ANDROIDDX-BindingHealth.ps1`	Read-only. Managed Google Play binding/connector health — bind status, owner, app-sync status/age, enabled paths
`ANDROIDDX-EndpointReachability.ps1`	Read-only. TCP reachability to Google/Microsoft endpoints from the device network; `-CompareConfig` diffs a dev-vs-prod snapshot
`ANDROIDDX-DiagnosticReport.ps1`	Read-only. Gathers tenant evidence and maps a chosen symptom to ranked likely causes (Playbook nodes); exports a config snapshot
`ANDROIDDX-Set-ScopeTargeting.ps1`	Default `-WhatIf`. Builds/refreshes the `Enrollment Scope – Android – Pilot/Expanded` Entra ID group (idempotent)

Named artifacts produced

- `Binding Health – Android – Baseline`
- `Enrollment Readiness – Android – Assessed`
- `Endpoint Allowlist – Android – Verified`
- `Config Baseline – Android – Dev` / `Config Baseline – Android – Prod`
- `Diagnostic Report – Android – <symptom>`
- `Repro Device – Android – Test`
- `Enrollment Scope – Android – Pilot` / `Enrollment Scope – Android – Expanded`

What the buyer receives

- This diagnostic playbook and its supporting guides as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP (read-only by default; `-WhatIf`-gated writes; no secrets).
- A repeatable process: Assess to Design to Configure to Pilot to Validate to Expand.

Outcome

Android enrollments that stall — accounts that won't add, dev-works-prod-fails, post-AMAPI breakage — root-caused by a repeatable decision tree and fixed, proven on a reproduction device and pilot before the fleet, with the AMAPI/re-enrollment points verified against current Microsoft guidance rather than assumed.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

ANDROIDDX-BindingHealth.ps1

Purpose: Reads the Android Enterprise (managed Google Play) connection settings from Microsoft Graph and reports the binding model, bind status, binding owner account, last app-sync time and status, and which device-management enrollment paths are enabled. Produces the 'Binding Health - Android - Baseline' artifact. Read-only — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-BindingHealth.ps1 -StaleHours 48 Requires the Microsoft Graph PowerShell SDK and the read scope DeviceManagementServiceConfig.Read.All (DeviceManagementConfiguration.Read.All also accepted by some tenants). EXPECTED RESULTS bindStatus='bound' and a recent successful app sync = a healthy connector. 'notBound', 'bindError', a missing owner account, or a stale/failed sync are the root cause behind binding/connector errors and app-delivery failures — take that node in the Diagnostic Playbook. .NOTES Android. Read-only against Microsoft Graph (beta resource). The androidManagedStoreAccountEnterpriseSettings resource is beta and evolving with the AMAPI transition — verify property names against current Microsoft Graph documentation for your tenant. > [CmdletBinding()] param([int]$StaleHours = 48, [string]$JsonPath)

How to run: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-BindingHealth.ps1 -StaleHours 48 Requires the Microsoft Graph PowerShell SDK and the read scope DeviceManagementServiceConfig.Read.All (DeviceManagementConfiguration.Read.All also accepted by some tenants). EXPECTED RESULTS bindStatus='bound' and a recent successful app sync = a healthy connector. 'notBound', 'bindError', a missing owner account, or a stale/failed sync are the root cause behind binding/connector errors and app-delivery failures — take that node in the Diagnostic Playbook. .NOTES Android. Read-only against Microsoft Graph (beta resource). The androidManagedStoreAccountEnterpriseSettings resource is beta and evolving with the AMAPI transition — verify property names against current Microsoft Graph documentation for your tenant. > [CmdletBinding()] param([int]$StaleHours = 48, [string]$JsonPath)

ANDROIDDX-DiagnosticReport.ps1

Purpose: Gathers the tenant-side evidence for an Android enrollment / "can't add a Google account" investigation — binding health, Android enrollment restriction, enrollment profiles, recent Android enrollments and their management agent — and maps the observed state to the most likely root-cause node in the Diagnostic Playbook. Produces the 'Diagnostic Report - Android - <symptom>' artifact and (with -JsonPath) a config snapshot you can diff dev-vs-prod. Read-only — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-DiagnosticReport.ps1 -Symptom CantAddAccount -JsonPath C:\Temp\prod-config.json -Symptom accepts: CantAddAccount | DevWorksProdFails | AfterAmapiMigration | BindingError | General Requires the Microsoft Graph PowerShell SDK and read scopes DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A ranked list of likely causes for the chosen symptom, each pointing to a Playbook node and its fix. Device-side signals (endpoint reachability, on-device account-add error) are gathered by ANDROIDDX-EndpointReachability.ps1 and the device logs — this report covers the tenant side. .NOTES Android. Read-only against Microsoft Graph (beta resources). Symptom->cause mapping is heuristic guidance, not an assertion — AMAPI/enrollment behavior is evolving; confirm each candidate against

current Microsoft guidance and your tenant before acting. > [CmdletBinding()] param([ValidateSet('CantAddAccount','DevWorksProdFails','AfterAmapiMigration','BindingError','General')] [string]$Symptom = 'General', [string]$JsonPath)

How to run: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-DiagnosticReport.ps1 -Symptom CantAddAccount -JsonPath C:\Temp\prod-config.json -Symptom accepts: CantAddAccount | DevWorksProdFails | AfterAmapiMigration | BindingError | General Requires the Microsoft Graph PowerShell SDK and read scopes DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A ranked list of likely causes for the chosen symptom, each pointing to a Playbook node and its fix. Device-side signals (endpoint reachability, on-device account-add error) are gathered by ANDROIDDX-EndpointReachability.ps1 and the device logs — this report covers the tenant side. .NOTES Android. Read-only against Microsoft Graph (beta resources). Symptom->cause mapping is heuristic guidance, not an assertion — AMAPI/enrollment behavior is evolving; confirm each candidate against current Microsoft guidance and your tenant before acting. > [CmdletBinding()] param([ValidateSet('CantAddAccount','DevWorksProdFails','AfterAmapiMigration','BindingError','General')] [string]$Symptom = 'General', [string]$JsonPath)

ANDROIDDX-EndpointReachability.ps1

Purpose: Two safe, read-only jobs behind one helper. (1) Default mode: from the network a device enrolls on, tests TCP 443 reachability to the managed Google Play / Android and Intune / Entra service endpoints that enrollment and "add a Google account" depend on, producing the 'Endpoint Allowlist - Android - Verified' artifact. (2) -CompareConfig mode: diffs two JSON config snapshots (e.g. a dev tenant vs a prod tenant) to expose the setting that differs when enrollment "works in dev but fails in prod". Nothing is written to any tenant. HOW TO RUN Reachability: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-EndpointReachability.ps1 -CsvPath C:\Temp\endpoints.csv Custom list: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-EndpointReachability.ps1 -Endpoints @('play.google.com','enrollment.manage.microsoft.com') Dev/Prod diff: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-EndpointReachability.ps1 -CompareConfig -DevJson .\dev-config.json -ProdJson .\prod-config.json Reachability mode needs no Graph connection. Snapshots for -CompareConfig are produced by ANDROIDDX-DiagnosticReport.ps1 -JsonPath (or any two comparable exports). EXPECTED RESULTS Every endpoint 'Reachable' = the network is not blocking enrollment/account-add. Any 'Blocked' is a proxy/firewall root cause. In -CompareConfig, the printed differences are the dev-vs-prod drift to reconcile. .NOTES Android. Read-only: network probes + local JSON diff; no tenant writes. The endpoint list below is a documented CORE set and is NOT exhaustive — Google and Microsoft publish and update the authoritative allowlists (Intune network endpoints; Android Enterprise network requirements). VERIFY against those current lists for your tenant/region before allowlisting. > [CmdletBinding()] param([string[]]$Endpoints, [int]$Port = 443, [string]$CsvPath, [switch]$CompareConfig, [string]$DevJson, [string]$ProdJson)

How to run: Reachability: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-EndpointReachability.ps1 -CsvPath C:\Temp\endpoints.csv Custom list: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-EndpointReachability.ps1 -Endpoints @('play.google.com','enrollment.manage.microsoft.com') Dev/Prod diff: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-EndpointReachability.ps1 -CompareConfig -DevJson .\dev-config.json -ProdJson .\prod-config.json Reachability mode needs no Graph connection. Snapshots for -CompareConfig are produced by ANDROIDDX-DiagnosticReport.ps1 -JsonPath (or any two comparable exports). EXPECTED RESULTS Every endpoint 'Reachable' = the network is not blocking

enrollment/account-add. Any 'Blocked' is a proxy/firewall root cause. In -CompareConfig, the printed differences are the dev-vs-prod drift to reconcile. .NOTES Android. Read-only: network probes + local JSON diff; no tenant writes. The endpoint list below is a documented CORE set and is NOT exhaustive — Google and Microsoft publish and update the authoritative allowlists (Intune network endpoints; Android Enterprise network requirements). VERIFY against those current lists for your tenant/region before allowlisting. > [CmdletBinding()] param([string[]]$Endpoints, [int]$Port = 443, [string]$CsvPath, [switch]$CompareConfig, [string]$DevJson, [string]$ProdJson)

ANDROIDDX-EnrollmentReadiness.ps1

Purpose: Builds the 'Enrollment Readiness - Android - Assessed' artifact. Reads the managed Google Play binding state, the Android platform enrollment restriction, any device-owner enrollment profiles, and recently-enrolled Android devices, then prints a readiness table with a PASS/WARN flag per check so you can tell before troubleshooting whether the tenant is even positioned to enroll. Read-only — it queries Microsoft Graph and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-EnrollmentReadiness.ps1 -Days 14 -CsvPath C:\Temp\android-readiness.csv Requires the Microsoft Graph PowerShell SDK and the read scopes DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. See ANDROIDDX-BindingHealth.ps1. EXPECTED RESULTS PASS on every check = tenant is positioned to enroll (binding bound, sync recent, Android enrollment allowed, at least one enrollment path configured). Any WARN points at the node in the Diagnostic Playbook to open next. Endpoint reachability is device-side — run ANDROIDDX-EndpointReachability.ps1 from the device network as well. .NOTES Android. Read-only against Microsoft Graph (beta resources noted inline). AMAPI transition is evolving — verify resource shapes against current Microsoft Graph documentation. > [CmdletBinding()] param([int]$Days = 14, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-EnrollmentReadiness.ps1 -Days 14 -CsvPath C:\Temp\android-readiness.csv Requires the Microsoft Graph PowerShell SDK and the read scopes DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. See ANDROIDDX-BindingHealth.ps1. EXPECTED RESULTS PASS on every check = tenant is positioned to enroll (binding bound, sync recent, Android enrollment allowed, at least one enrollment path configured). Any WARN points at the node in the Diagnostic Playbook to open next. Endpoint reachability is device-side — run ANDROIDDX-EndpointReachability.ps1 from the device network as well. .NOTES Android. Read-only against Microsoft Graph (beta resources noted inline). AMAPI transition is evolving — verify resource shapes against current Microsoft Graph documentation. > [CmdletBinding()] param([int]$Days = 14, [string]$CsvPath)

ANDROIDDX-Set-ScopeTargeting.ps1

Purpose: Creates (if missing) and populates the security group that scopes an enrollment troubleshooting pilot — e.g. 'Enrollment Scope - Android - Pilot' for the reproduction/pilot set or 'Enrollment Scope - Android - Expanded' for the fleet. Members are added from a CSV of user principal names so every retest and every rollout targets a known, repeatable set. Writes group membership only; -WhatIf shows the plan without changing anything. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-Set-ScopeTargeting.ps1 -GroupName "Enrollment Scope - Android - Pilot" -MembersCsv .\pilot-users.csv -WhatIf Apply: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-Set-ScopeTargeting.ps1 -GroupName "Enrollment Scope - Android - Pilot" -MembersCsv .\pilot-users.csv MembersCsv expects a single column: UserPrincipalName. Requires the Microsoft Graph

SDK and scopes Group.ReadWrite.All, User.Read.All. Assign your Android enrollment profile/config to this group in Intune. EXPECTED RESULTS The named group exists and contains the listed users; new members are added, existing ones left in place (idempotent). Use it as 'Enrollment Scope - Android - Pilot/Expanded' for assignment. .NOTES Android. Writes Entra ID group membership; -WhatIf-safe and idempotent. No credentials/secrets in code; least-privilege scopes only. > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)][string]$GroupName, [Parameter(Mandatory)][string]$MembersCsv) if (-not (Test-Path $MembersCsv)) { throw "MembersCsv not found: $MembersCsv (column: UserPrincipalName)." } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','User.Read.All' -NoWelcome

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-Set-ScopeTargeting.ps1 -GroupName "Enrollment Scope - Android - Pilot" -MembersCsv .\pilot-users.csv -WhatIf Apply: powershell -ExecutionPolicy Bypass -File .\ANDROIDDX-Set-ScopeTargeting.ps1 -GroupName "Enrollment Scope - Android - Pilot" -MembersCsv .\pilot-users.csv MembersCsv expects a single column: UserPrincipalName. Requires the Microsoft Graph SDK and scopes Group.ReadWrite.All, User.Read.All. Assign your Android enrollment profile/config to this group in Intune. EXPECTED RESULTS The named group exists and contains the listed users; new members are added, existing ones left in place (idempotent). Use it as 'Enrollment Scope - Android - Pilot/Expanded' for assignment. .NOTES Android. Writes Entra ID group membership; -WhatIf-safe and idempotent. No credentials/secrets in code; least-privilege scopes only. > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)][string]$GroupName, [Parameter(Mandatory)][string]$MembersCsv) if (-not (Test-Path $MembersCsv)) { throw "MembersCsv not found: $MembersCsv (column: UserPrincipalName)." } if (-not (Get-Module -ListAvailable -Name Microsoft.Graph.Authentication)) { Install-Module Microsoft.Graph -Scope CurrentUser -Force -AllowClobber } Import-Module Microsoft.Graph.Authentication -ErrorAction Stop Connect-MgGraph -Scopes 'Group.ReadWrite.All','User.Read.All' -NoWelcome